

TALLER ANALÍTICO: CONTANDO PARÁMETROS (25 MIN)

Diseñemos la arquitectura de una IA para aprobar tarjetas de crédito. Tenemos **3 entradas** (Edad, Ingresos, Deuda). Decidimos poner **1 Capa Oculta con 4 neuronas**. Finalmente, **1 Capa de Salida con 1 neurona** (Aprobado/Rechazado).

1. Calcule cuántos **Pesos (W)** hay entre la Capa de Entrada y la Capa Oculta.
(Recuerde que cada entrada se conecta a cada neurona oculta).
2. Cada neurona oculta tiene 1 Sesgo (Bias). ¿Cuántos sesgos hay en esa capa?
3. Calcule los Pesos (W) y el Sesgo (b) entre la Capa Oculta y la Capa de Salida.
4. Suma todo. ¿Cuántos "parámetros entrenables" tiene esta pequeña red neuronal en total?

1. Como cada entrada se conecta con cada neurona oculta, simplemente se multiplican:
 $3 \text{ entradas} \times 4 \text{ neuronas} = 12 \text{ pesos}$
2. Como cada neurona oculta tiene su propio sesgo, y hay 4 neuronas:
Hay 4 sesgos.
3. Ahora las 4 neuronas ocultas se conectan con la única neurona de salida:
 $4 \text{ neuronas ocultas} \times 1 \text{ neurona de salida} = 4 \text{ pesos}$
Y como la capa de salida tiene 1 sola neurona, tiene 1 solo sesgo:
Hay 4 pesos y 1 sesgo.
4. $12 \text{ pesos (entrada - oculta)} + 4 \text{ sesgos (capa oculta)} + 4 \text{ pesos (oculta - salida)} + 1 \text{ sesgo (capa de salida)} = 21 \text{ parámetros en total}$

Esta pequeña red tiene 21 parámetros entrenables. Esto significa que la red tiene 21 "perillas" que puede ir ajustando por sí sola durante el entrenamiento hasta aprender a aprobar o rechazar tarjetas de crédito correctamente

TALLER DE LABORATORIO: EXPLORANDO LAS MATRICES (35 MIN)**Misión Práctica:**

1. Copien el código en su IDE y ejecútenlo. Observen el valor de probabilidad.
2. Impriman la variable Z1 y A1. Analicen cómo la función sigmoide transformó los valores puros de Z1 al rango (0, 1).
3. **El Reto Dimensional:** Modifiquen el código para que la red procese **2 clientes al mismo tiempo** (procesamiento en Lote / Batch). Cambien la variable X a una matriz de 2x3: `np.array([[0.5, 0.8, 0.2], [0.1, 0.9, 0.9]])`.
4. Ejecuten el código. Si usaron `np.dot` correctamente, el álgebra lineal calculará las salidas de los dos clientes simultáneamente sin necesidad de cambiar las matrices de pesos. ¡Este es el poder del cálculo tensorial!

```

8     [0.1, 0.2, -0.3, 0.4],
9     [-0.5, 0.6, 0.7, -0.8],
10    [0.9, -0.1, 0.2, 0.3]
11 ])
12 b1 = np.array([0.1, -0.2, 0.3, -0.4])
13
14 Z1 = np.dot(X, W1) + b1
15 A1 = sigmoide(Z1)
16
17 print("Z1 (valores crudos, sin transformar):")
18 print(Z1)
19 print("\nA1 (después de la sigmoide, entre 0 y 1):")
20 print(A1)

```

Problems Output Debug Console Terminal Ports

```

@gomezzuley19-debug →/workspaces/IA-2_-Talleres (main) $ source /workspaces/IA-2_-Talleres/.venv/bin/activate
(.venv) @gomezzuley19-debug →/workspaces/IA-2_-Talleres (main) $ /workspaces/IA-2_-Talleres/.venv/bin/python "/workspaces/IA-2_-Talleres/main.py"
Z1 (valores crudos, sin transformar):
[-0.07  0.36  0.75 -0.78]

A1 (después de la sigmoide, entre 0 y 1):
[0.48250714 0.58904043 0.6791787 0.31431989]

Predicción de la Red (Probabilidad): 0.6259
(.venv) @gomezzuley19-debug →/workspaces/IA-2_-Talleres (main) $

```

Al comparar Z1 con A1 se nota claramente el trabajo de la función sigmoide. Los valores de Z1 son números "crudos" que salen directamente de la multiplicación: pueden ser negativos, positivos, grandes o pequeños, sin ningún límite. En cambio, los valores de A1 siempre quedan entre 0 y 1, sin importar qué tan grandes o pequeños fueran los originales.

La sigmoide funciona como un "aplastador": toma cualquier número y lo comprime dentro de ese rango. Si el valor de Z1 era negativo, la sigmoide lo convierte en algo cercano a 0. Si era positivo, lo acerca a 1. Y si era cero, da exactamente 0.5 (justo en la mitad).

```

1  import numpy as np
2
3  def sigmoide(x):
4      return 1 / (1 + np.exp(-x))
5
6  X = np.array([
7      [0.5, 0.8, 0.2],
8      [0.1, 0.9, 0.9]
9  ])
10
11  W1 = np.array([
12      [0.1, 0.2, -0.3, 0.4],
13      [-0.5, 0.6, 0.7, -0.8],
14      [0.9, -0.1, 0.2, 0.3]
15  ])
16  b1 = np.array([0.1, -0.2, 0.3, -0.4])
17
18  Z1 = np.dot(X, W1) + b1

```

Problems Output Debug Console Terminal Ports 1

```

• (.venv) @gomezzuley19-debug → /workspaces/IA-2_-Talleres (main) $ /workspaces/IA-2_-Talleres
Z1 (ahora tiene 2 filas, una por cliente):
[[-0.07  0.36  0.75 -0.78]
 [ 0.47  0.27  1.08 -0.81]]

A1:
[[0.48250714 0.58904043 0.6791787  0.31431989]
 [0.61538376 0.5670929  0.74649398 0.3078905 ]]

Predicción para el Cliente 1: 0.6259
Predicción para el Cliente 2: 0.6539
○ (.venv) @gomezzuley19-debug → /workspaces/IA-2_-Talleres (main) $ 

```

Lo más interesante de este ejercicio es que, para procesar dos clientes en vez de uno, no fue necesario cambiar absolutamente nada de la red: ni los pesos, ni los sesgos, ni las operaciones. Lo único que cambió fue la forma de los datos de entrada, que pasó de ser una lista de 3 valores a una matriz de 2 filas por 3 columnas.

NumPy se encargó automáticamente de hacer los cálculos para ambos clientes al mismo tiempo, y el resultado final ya no fue un solo número, sino dos predicciones (una por cliente). Esto ocurre porque la multiplicación de matrices está diseñada para trabajar con varias filas a la vez de forma natural.

Esta es precisamente la razón por la que las redes neuronales modernas pueden procesar miles de imágenes o datos simultáneamente: en lugar de repetir el mismo cálculo uno por uno con ciclos, se apilan todos los datos en una gran matriz y se hace una sola operación matemática que los resuelve todos de golpe. Es lo que hace que entrenar modelos de Deep Learning sea posible en un tiempo razonable.